

1. (Question 1) The Jupyter notebook file *Q1.ipynb* has all relevant code for this problem. Codex GPT-6 Astra assisted me in writing the code. After implementing a basic euclidean distance function, I created a K-nearest neighbors classifier that used the provided training file to predict on the test file for $k = 1 \dots 9$. Note that in the definition of the knn_classifier, ties are decided by choosing the label of the nearest neighbor since the labels array is sorted by increasing distance from least to greatest, and the max would return the first label in the case of a tie.

```
def euclidean_distance(dimension, point1, point2):
    squared_distance = 0
    for i in range(dimension):
        squared_distance += (point1[i] - point2[i]) ** 2
    return squared_distance ** 0.5

def knn_classifier(k, training_data, test_point):
    distances = []
    for data_point in training_data:
        distance = euclidean_distance(len(test_point), data_point
[: -1], test_point)
        distances.append((distance, data_point[-1]))
    distances.sort(key=lambda x: x[0])
    k_nearest_neighbors = distances[:k]
    labels = [neighbor[1] for neighbor in k_nearest_neighbors]
    return max(labels, key=labels.count)
```

I got the following results, which are also in the notebook file:

kNN results on the original, unscaled data

K	1	2	3	4	5	6	7	8	9
Correct predictions	462	462	462	462	462	462	462	462	462
Accuracy	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000

I was surprised that the accuracy was 100% for all k -values and I was concerned that I had implemented the algorithm incorrectly. But after reviewing my implementation, I think it is correct. Next, I implemented min-max normalization and standardization to scale the data and then re-ran my KNN algorithm. In both cases, if a certain feature in the training data had zero variance or was constant, I set that feature to 0.

```

def min_max_normalize(data, feature_ranges=None):
    # When no ranges are supplied, calculate them from the training data.
    if feature_ranges is None:
        feature_ranges = []
        for i in range(len(data[0]) - 1): # Exclude the final label column.
            column = [row[i] for row in data]
            feature_ranges.append((min(column), max(column)))

    normalized_data = []
    for row in data:
        normalized_row = []
        for i in range(len(row) - 1):
            min_value, max_value = feature_ranges[i]
            if max_value == min_value:
                normalized_value = 0.0 # Ignore a constant training feature
            else:
                normalized_value = (row[i] - min_value) / (max_value -
min_value)
            normalized_row.append(normalized_value)
        normalized_row.append(row[-1]) # Keep the original class label.
        normalized_data.append(normalized_row)

    return normalized_data, feature_ranges

```

```

def standardize(data, feature_stats=None):
    # When no stats are supplied, calculate them from the training data.
    if feature_stats is None:
        feature_stats = []
        for i in range(len(data[0]) - 1): # Exclude the final label column.
            column = [row[i] for row in data]
            mean = sum(column) / len(column)
            variance = sum((x - mean) ** 2 for x in column) / len(column)
            std_dev = variance ** 0.5
            feature_stats.append((mean, std_dev))

    standardized_data = []
    for row in data:
        standardized_row = []
        for i in range(len(row) - 1):
            mean, std_dev = feature_stats[i]
            if std_dev == 0:
                standardized_value = 0.0 # Ignore a constant training
feature.
            else:
                standardized_value = (row[i] - mean) / std_dev
            standardized_row.append(standardized_value)
        standardized_row.append(row[-1]) # Keep the original class label.
        standardized_data.append(standardized_row)

    return standardized_data, feature_stats

```

I re-ran the KNN classifier on the two versions of the scaled data and got the following results:

Min-max normalization									
<i>K</i>	1	2	3	4	5	6	7	8	9
Correct predictions	461	461	461	461	461	461	461	461	461
Accuracy	0.998	0.998	0.998	0.998	0.998	0.998	0.998	0.998	0.998

Standardization									
<i>K</i>	1	2	3	4	5	6	7	8	9
Correct predictions	461	461	461	461	461	461	461	461	461
Accuracy	0.998	0.998	0.998	0.998	0.998	0.998	0.998	0.998	0.998

Both scaling methods resulted in a slight decrease in accuracy, misclassifying only one test point. I was again suprised because I had expected that scaling would only improve accuracy, or at least not hurt it. I was again worried that I had implemented the algorithm incorrectly, but I cannot identify what is wrong with the implementation so I am working on the assumption that the given data has been set up this way on purpose and that my implementation is correct.

Given these results, it would appear that the original unscaled data is the best approach for this particular dataset, but that is not necessarily true in general. I wanted to know which point was misclassified, so the additional code in the file was created by Codex to help me identify which point label was changed after scaling.

In general, I would expect scaling to improve accuracy for the KNN algorithm so that no one feature has an outsized influence on the calcuated distance, but if one feature already has low variance, it's change from the original data to scaling might mean that after scaling it can influence the classification. Feature 4 from the csv appears to do that in this case. I became curious about whether we could drop a feature entirely and still perfectly predict the test set. I prompted Codex to check all possible subsets of the features and run the KNN algorithm for the original data and the two different scaled data. This code showed that if we dropped Feature 4 and used the standardized data, we could stil get 100% accuracy for $k = 1...9$. I think this supports my hypothesis that Feature 4 is the one that is causing the misclassification after scaling.

2. (Question 2)